

第九部分 数学问题

9.1 多项式展开系数

源程序名	equal.??? (pas, c, cpp)
可执行文件名	equal.exe
输入文件名	equal.in
输出文件名	equal.out

【问题描述】

二项式展开系数大家已经十分熟悉了：

$$(x+y)^n = \sum_{i=0}^n C_n^i x^i y^{n-i}$$

现在我们将问题推广到任意 t 个实数的和的 n 次方 $(x_1+x_2+\cdots+x_t)^n$ 的展开式。我们想知道多项式 $(x_1+x_2+\cdots+x_t)^n$ 中的任意一项 $x_1^{n_1} x_2^{n_2} \cdots x_t^{n_t}$ 的系数。例如，将一个三项式 $(x_1+x_2+x_3)^3$ 展开后，可以得到：

$$(x_1+x_2+x_3)^3 = x_1^3 + x_2^3 + x_3^3 + 3x_1^2x_2 + 3x_1^2x_3 + 3x_1x_2^2 + 3x_1x_3^2 + 3x_2^2x_3 + 3x_2x_3^2 + 6x_1x_2x_3 \quad \text{其中，}$$

$x_1^2x_2$ 的系数为 3

【输入】

第一行，两个整数 n 和 t ，中间用空格分隔。分别表示多项式幂和项数。

第二行， t 个整数 $n_1, n_2, \cdots, n_t$ ，中间用空格分隔。分别表示 $x_1, x_2, \cdots, x_t$ 的幂。 $(n_1+n_2+\cdots+n_t=n, 1 \leq n, t \leq 12)$

【输出】

仅一行，一个整数（保证在长整型范围内）。表示多项式 $(x_1+x_2+\cdots+x_t)^n$ 中的项 $x_1^{n_1} x_2^{n_2} \cdots x_t^{n_t}$ 的系数。

【样例】

equal.in	equal.out
3 3	3
2 1 0	

【知识准备】

组合数运算；二项式展开；代数式的恒等变形。

【算法分析】

(1) 利用二项式展开公式 $(x+y)^n = \sum_{i=0}^n C_n^i x^i y^{n-i}$ 推广到多项式

$(x_1+x_2+\cdots+x_t)^n$ 的展开：

$$\begin{aligned}
(x_1 + x_2 + \dots + x_t)^n &= \sum_{k_1=0}^n C_n^{k_1} x_1^{k_1} (x_2 + x_3 + \dots + x_t)^{n-k_1} \\
&= \sum_{k_1=0}^n C_n^{k_1} x_1^{k_1} \cdot \sum_{k_2=0}^{n-k_1} C_{n-k_1}^{k_2} x_2^{k_2} (x_3 + x_4 + \dots + x_t)^{n-k_1-k_2} \\
&= \dots\dots \\
&= \sum_{k_1=0}^n C_n^{k_1} x_1^{k_1} \cdot \sum_{k_2=0}^{n-k_1} C_{n-k_1}^{k_2} x_2^{k_2} \cdot \dots \cdot \sum_{k_{t-1}=0}^{n-k_1-k_2-\dots-k_{t-2}} C_{n-k_1-k_2-\dots-k_{t-2}}^{k_{t-1}} x_{t-1}^{k_{t-1}} \cdot \sum_{k_t=0}^{n_t} C_{n-k_1-k_2-\dots-k_{t-1}}^{k_t} x_t^{k_t}
\end{aligned}$$

其中含有 $x_1^{n_1} x_2^{n_2} \dots x_t^{n_t} (n_1 + n_2 + \dots + n_t = n)$ 的一项即为上式右边展开后的一项：

$$C_n^{n_1} x_1^{n_1} \cdot C_{n-n_1}^{n_2} x_2^{n_2} \cdot \dots \cdot C_{n-n_1-n_2-\dots-n_{t-2}}^{n_{t-1}} x_{t-1}^{n_{t-1}} \cdot C_{n_t}^{n_t} x_t^{n_t}$$

$$\text{整理得到： } C_n^{n_1} \cdot C_{n-n_1}^{n_2} \cdot \dots \cdot C_{n-n_1-n_2-\dots-n_{t-2}}^{n_{t-1}} \cdot C_{n_t}^{n_t} \cdot x_1^{n_1} x_2^{n_2} \dots x_{t-1}^{n_{t-1}} x_t^{n_t} \quad \text{①}$$

所以要想求 $(x_1+x_2+\dots+x_t)^n$ 的任意一项 $x_1^{n_1} x_2^{n_2} \dots x_t^{n_t}$ 的系数只要将①式中令 $x_1=x_2=\dots$

$$=x_t=1, \text{ 得到 } C_n^{n_1} \cdot C_{n-n_1}^{n_2} \cdot \dots \cdot C_{n-n_1-n_2-\dots-n_{t-2}}^{n_{t-1}} \cdot C_{n_t}^{n_t}$$

(2) 因为

$$C_n^{n_1} = \frac{n!}{n_1!(n-n_1)!}$$

$$C_{n-n_1}^{n_2} = \frac{(n-n_1)!}{n_2!(n-n_1-n_2)!}$$

.....

$$C_{n-n_1-n_2-\dots-n_{t-2}}^{n_{t-1}} = \frac{(n-n_1-n_2-\dots-n_{t-2})!}{n_{t-1}!(n-n_1-n_2-\dots-n_{t-2}-n_{t-1})!} = \frac{(n-n_1-n_2-\dots-n_{t-2})!}{n_{t-1}!n_t!}$$

$$C_{n_t}^{n_t} = \frac{n_t!}{n_t!}$$

$$\text{所以 } C_n^{n_1} \cdot C_{n-n_1}^{n_2} \cdot \dots \cdot C_{n-n_1-n_2-\dots-n_{t-2}}^{n_{t-1}} \cdot C_{n_t}^{n_t} = \frac{n!}{n_1!n_2!\dots n_{t-1}!n_t!}$$

(3) 由分析 (1)、(2) 求 $x_1^{n_1} x_2^{n_2} \dots x_t^{n_t}$ 的系数化为求上述的除法运算：

- ① 先计算 $n!$;
- ② 将 $n!$ 逐一去除 $n_1, n_2, \dots, n_t$;
- ③ 最后的商为所求结果。

由于题目所给 $n \leq 12$, 计算 $n!$ 可在长整型范围内, 无需采用高精度运算。

设数组 `int no[1..t]nt`; `total` 为 $n!$ 。主要算法描述如下

```

total=1;
for(i=1;i<=n;i++) total=total+i;      {求 n!}
for(i=1;i<=t;i++)
    for(j=1;j<=no[i];j++)
        total=total / j;  {整除 n!}

```

9.2 两数之和

源程序名	pair.??? (pas, c, cpp)
可执行文件名	pair.exe
输入文件名	pair.in
输出文件名	pair.out

【问题描述】

我们知道从 n 个非负整数中任取两个相加共有 $n*(n-1)/2$ 个和，现在已知这 $n*(n-1)/2$ 个和值，要求 n 个非负整数。

【输入】

输入文件仅有一行，包含 $n*(n-1)/2+1$ 个空格隔开的非负整数，其中第一个数表示 n ($2 < n < 10$)，其余 $n*(n-1)/2$ 个数表示和值，每个数不超过 100000。

【输出】

输出文件仅一行，按从小到大的次序依次输出一组满足要求的 n 个非负整数，相邻两个整数之间用一个空格隔开；若问题无解则输出 “Impossible”。

【样例】

pair.in	pair.out
3 1269 1160 1663	383 777 886

【算法分析】

本题的规模只有 10，看似一道搜索题。其实不然，搜索固然有可能出解，但是本题是有多项式级算法的。本题的难点就在于要理清这看似混乱的 $n(n-1)/2$ 个“和”的关系，从中寻找突破口。

那么，突破口在哪里呢？ $n(n-1)/2$ 个“和”是任意给出的，并未按照什么指定的顺序，所以从表面上看这些数（和）之间似乎没有什么区别。当然，我们必须在这些数（和）中制造出一些区别来，因为没有区别我们也就无从下手。对什么信息也没有的一些数，惟一可以制造出的区别就是它们之间的大小关系。根据数的大小关系，我们可以对这 $n(n-1)/2$ 个数排序。排完序后，可以成为突破口的无非就是那么几个，最小的数（和）、最大的数（和）、中位数……我们不可能随便找一个数出来作为突破口，因为随便找一个数的话，大小关系就失去意义了。所以，这时我们需要注意的只有这么几个数了：最小的数（和）、最大的数（和）和中位数。

中位数看上去似乎也很难成为突破口，因为我们无法预知中位数是哪两个数的和。而最小、最大的两个数（“最小的和”与“最大的和”）则不同，最小的和必然是最小的两个数之和，最大的和必然是最大的两个数之和。由问题对称性，最小与最大其实是等价的，所以我们可以只考虑最小的情况，解决了最小，最大也就相应解决了。

最小的和是最小的两个数之和，这是一条已经确定的与最小的两个数相关的信息。当然，就凭这些信息，我们还是无法知道最小的两个数究竟是多少，这是我们目前面临的一个障碍。我们不妨先跳过这个障碍，考虑以后的问题。我们可以假设已经知道最小的数的大小，这样

我们又可以很顺利的知道次小的数是多少了（最小的和-最小的数）。

知道最小的两个数的大小对我们有什么好处呢？其实关键是知道最小的数是多少，这是至关重要的，因为很多特殊的和都与最小的数有关。试想，我们知道了最小的两个数，也就确定了一个和（最小的和）。把这个“和”从 $n(n-1)/2$ 个和中去掉，剩下的 $n(n-1)/2-1$ 个和中又会产生新的最小的和。这个新的最小的和是哪两个数的和呢？应该是最小的数与第三小的数的和。由于我们假设已经知道最小的数，我们自然就可以通过作差得到第三小的数了。

得到前三小的数后，我们就确定了 3 个“和”了。将这三个“和”从 $n(n-1)/2$ 个和中去掉，剩下的 $n(n-1)/2-3$ 个和中又会产生新的最小的和。这个新的最小的和必然是最小的数与第四小的数的和。由于最小的数已知，第四小的数就可以算出了。

更一般的情况，知道了前 k 小的数，那么我们就确定了 $k(k-1)/2$ 个和了。将这 $k(k-1)/2$ 个和从 $n(n-1)/2$ 个和中去掉，剩下的 $n(n-1)/2-k(k-1)/2$ 个和中又会产生新的最小的和。这个新的最小的和必然是最小的数与第 $k+1$ 小的数的和。由于最小的数已知，第 $k+1$ 小的数就可以通过作差得到。

如此，在知道了最小的数的情况下，不断的从当前最小的和入手，就可以从小到大依次把所有的数都推算出来了。

不过，有一点还是需要注意的，前面所有的推导都是建立在最小的数已知的情况下的，但实际情况是最小的数未知。所以，我们还需要创造条件，使最小的数由未知变为已知。

一个简单而可行的方法就是枚举最小的数。简单就不必说了，为什么说枚举是可行的呢？因为题目规定所有的数都不超过 100000，因此总共需要枚举的方案数最多只有 100000。而在知道最小的数的情况下，推算出其他数的代价是 $O(n^2)$ ， $n \leq 10$ 。100000*10² 的计算代价是可以承受的。

最后，我们总结一下前面得到的算法。

- (1) 枚举最小的数；
- (2) 根据最小的数和最小的和，得到次小的数；
- (3) 由前 k 小的数推出第 $k+1$ 小的数。

算法的时间复杂度是 $O(Cn^2)$ ，其中 C 表示数值的大小。

从本题的解题过程中，我们看到其中最关键的一步就是以最小的和作为突破口。在解决数学问题的时候，很多情况下从为数不多的信息中寻找突破口是至关重要的。找到了突破口，问题本身也就迎刃而解了。

9.3 盒子与球

源程序名	box.??? (pas, c, cpp)
可执行文件名	box.exe
输入文件名	box.in
输出文件名	box.out

【问题描述】

现有 r 个互不相同的盒子和 n 个互不相同的球，要将这 n 个球放入 r 个盒子中，且不允许有空盒子。问有多少种方法？

例如：有 2 个不同的盒子（分别编为 1 号和 2 号）和 3 个不同的球（分别编为 1、2、3 号），则有 6 种不同的方法：

1 号盒子	1 号球	1、2 号球	1、3 号球	2 号球	2、3 号球	3 号球
2 号盒子	2、3 号球	3 号球	2 号球	1、3 号球	1 号球	1、2 号球

【输入】

两个整数， n 和 r ，中间用空格分隔。（ $0 \leq n, r \leq 10$ ）

【输出】

仅一行，一个整数（保证在长整型范围内）。表示 n 个球放入 r 个盒子的方法。

【样例】

box.in
3 2
box.out
6

【知识准备】

第二类 Stirling 数。

【算法分析】

先考虑三种情况：

- （1）若盒子数 r 大于球数 n ，根据题意不允许有空盒的要求，放置的方法数为零；
- （2）若 $r=1$ ，则只有一种放法；
- （3）若 $r=n$ ，相当于将 n 个不同的球进行排列，故有 $n!$ 种。

下面考虑 $1 < r < n$ 的情况。

n 个不同的球放入 r 个不同的盒子中，可以先把 r 个盒子视为相同的，即先求出 n 个不同的球放入 r 个相同的盒子中，且不允许有空盒子的不同方案数，设为 $S(n, r)$ ；再将 r 个盒子进行全排列，共有 $r!$ 。

记号 $f(n, r)$ 为 n 个不同的球放入 r 个不同盒子中，且不允许有空盒子的不同放法，根据乘法原理，有 $f(n, r) = S(n, r) \cdot r!$

如何求 $S(n, r)$ ？请看如下定义：

定义 n 个有区别的球放入 r 个相同的盒子中，要求无一空盒，其不同的方案数 $s(n, r)$ 称为第二类 Stirling 数。

例如：红(k)、黄(y)、蓝(b)、白(w)四种颜色的球，放入两个无区别的盒子里，不允许空盒，其方案数共有如下 7 种：

方案	1	2	3	4	5	6	7
----	---	---	---	---	---	---	---

第 1 盒子	r	y	b	w	ry	rb	rw
第 2 盒子	ybw	rbw	ryw	ryb	bw	yw	yb

所以 $S(4, 2)=7$

下面就 $1 \leq n \leq 5$, $1 \leq r \leq n$ 列出各个 $S(n, r)$ 的植如下

$n \backslash r$	1	2	3	4	5
1	1				
2	1	1			
3	1	3	1		
4	1	7	6	1	
5	1	15	25	10	1

定理 第二类 Stirling 数满足下面的递推关系

$$S(n, r) = r \cdot S(n-1, r) + S(n-1, r-1) \quad (n > 1, r \geq 1)$$

证明：设 n 个不同的球为 $b_1, b_2, \dots, b_n$ ，从中取一个球设为 b_1 。把这 n 个球放入 r 个盒子无一空盒的方案全体可分为两类：

(1) b_1 独占一盒，其余 $n-1$ 个球放入 $r-1$ 个盒子无一空盒的方案数为 $S(n-1, r-1)$

(2) b_1 不独占一盒，这相当于先将 $b_2, b_3, \dots, b_n$ 。这 $n-1$ 个球放入 r 个盒子，不允许有空盒的方案数共有 $S(n-1, r)$ ，然后将 b_1 放入其中一盒，这一盒子有 r 种可挑选，故 b_1 不独占一盒的方案数为 $r \cdot S(n-1, r)$ 。

根据加法原理，则有：

$$S(n, r) = r \cdot S(n-1, r) + S(n-1, r-1) \quad (n > 1, r \geq 1)$$

对于 $n=r$ ，或 $r=1$ ，显然有 $S(n, n)=1$, $S(n, 1)=1$ 。

综上所述，本题的算法可用第二类 Stirling 数的递推关系先求出 $S(n, r)$ ，再乘上 $r!$ ，即得所求方案数。

主要算法描述如下：

(1) 数据结构

`const int maxn=10; {球的最大数目}`

`int s[maxn][maxn]; {存放第二类数}`

(2) 用下列二重循环求 $S(n, r)$

① 置 $S[0][0]=1$;

② `for(i=1;i<=n;i++)`

`for(j=1;j<=r;j++)`

`S(i, j)=i*S(i-1, j)+S(i-1, j-1)`

③ 计算方案数 S

`ANS=S[n][r]`

`for(i=1;i<=r;i++)ANS=ANS*i;`

9.4 取数游戏

源程序名	cycle.??? (pas, c, cpp)
可执行文件名	cycle.exe
输入文件名	cycle.in
输出文件名	cycle.out

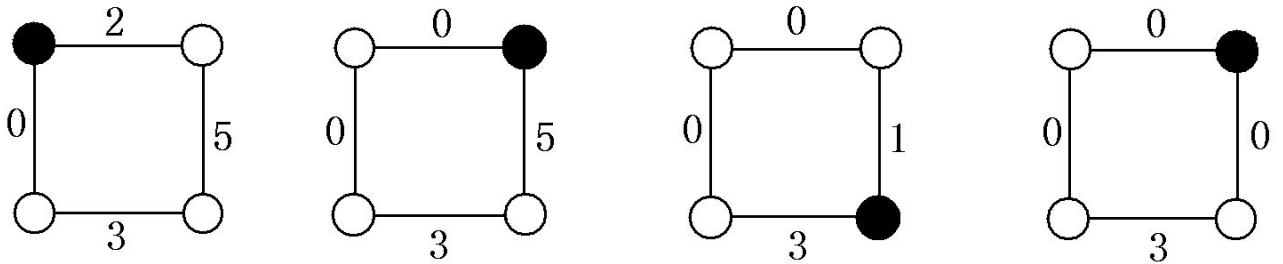
【问题描述】

有一个取数的游戏。初始时，给出一个环，环上的每条边上都有一个非负整数。这些整数中至少有一个 0。然后，将一枚硬币放在环上的一个节点上。两个玩家就是以这个放硬币的节点为起点开始这个游戏，两人轮流取数，取数的规则如下：

- (1) 选择硬币左边或者右边的一条边，并且边上的数非 0；
- (2) 将这条边上的数减至任意一个非负整数(至少要有所减小)；
- (3) 将硬币移至边的另一端。

如果轮到一个人走，这时硬币左右两边的边上的数值都是 0，那么这个玩家就输了。

如下图，描述的是 Alice 和 Bob 两人的对弈过程，其中黑色节点表示硬币所在节点。结果图 (d) 中，轮到 Bob 走时，硬币两边的边上都是 0，所以 Alice 获胜。



(a) Alice

(b) Bob

(c) Alice

(d) Bob

现在，你的任务就是根据给出的环、边上的数值以及起点（硬币所在位置），判断先走方是否有必胜的策略。

【输入】

第一行一个整数 N ($N \leq 20$)，表示环上的节点数。

第二行 N 个数，数值不超过 30，依次表示 N 条边上的数值。硬币的起始位置在第一条边与最后一条边之间的节点上。

【输出】

仅一行。若存在必胜策略，则输出 “YES”，否则输出 “NO”。

【样例】

cycle.in	cycle.out
4	YES
2 5 3 0	
cycle.in	cycle.out
3	NO
0 0 0	

【问题分析】

本题的数据规模 ($N \leq 20$, 边上的数值不超过 30) 实际上是一个幌子, 是想误导我们走向搜索的道路。

考虑一个简化的问题。如果硬币的一边的数值为 0, 那么惟一可能取胜的走法就是向另一边移动, 并且把边上的数减到 0。因为如果不把边上的数减到 0, 那么下一步对方会将硬币移动到原来的位置, 并且将边上的数减到 0, 这样硬币两边的数值就都为 0 了。所以, 对于一边有 0 的情况, 双方惟一的走法就是不停的向另一边移动, 并取完边上的数值。因此, 判断是否有必胜策略, 就是看另一个方向上连续的非零边是否为奇数条。

那么两边都非零的情况呢? 如果有一个方向上连续的非零边为奇数条, 那么显然是有必胜策略的, 因为只需往这个方向走并取完边上的数即可。如果两个方向上连续的非零边都是偶数条, 则没有必胜策略。因为不管往哪个方向走, 必然不能取完边上的数, 否则必败。如果不取完, 则下一步对方可以将硬币移动回原来的位置并取完边上的数, 这样就变成了一边为 0、另一边有偶数条连续的非零边的情况, 还是必败。所以, 对于一般的情况, 只需判断硬币的两边是否至少有一边存在奇数条连续的非零边。如果存在, 则有必胜策略; 否则无必胜策略。算法的时间复杂度为 $O(N)$ 。

9.5 磁盘碎片整理

源程序名	defrag.??? (pas, c, cpp)
可执行文件名	defrag.exe
输入文件名	defrag.in
输出文件名	defrag.out

【问题描述】

出于最高安全性考虑，司令部采用了特殊的安全操作系统，该系统采用一个特殊的文件系统。在这个文件系统中所有磁盘空间都被分成了相同尺寸的 N 块，用整数 1 到 N 标识。每个文件占用磁盘上任意区域的一块或多块存储区，未被文件占用的存储块被认为是可是用的。如果文件存储在磁盘上自然连续的存储块中，则能被以最快的速度读出。

因为磁盘是匀速转动的，所以存取上面不同的存储块需要的时间也不同。读取磁盘开头处的存储块比读取磁盘尾处的存储块快。根据以上现象，我们事先将文件按其存取频率的大小用整数 1 到 K 标识。按文件在磁盘上的最佳存储方法，1 号文件将占用 1, 2, ..., S_1 的存储块，2 号文件将占用 $S_1+1, S_1+2, \dots, S_1+S_2$ 的存储块，以此类推 (S_i 是被第 i 个文件占用的存储块的个数)。为了将文件以最佳形式存储在磁盘上，需要执行存储块移动操作。一个存储块移动操作包括从磁盘上读取一个被占用的存储块至内存并将它写入其他空的存储块，然后宣称前一个存储块被释放，后一个存储块被占用。

本程序的目的是通过执行最少次数的存储块移动操作，将文件按最佳方式存储到磁盘上，注意同一个文件的存储块在移动之后其相对次序不可改变。

【输入】

每个磁盘说明的第一行包含两个用空格隔开的整数 N 和 K ($1 \leq K \leq N \leq 100000$)，接下来的 K 行每行说明一个文件，对第 i 个文件的说明是这样的：首先以整数 S_i 开头，表示第 i 个文件的存储块数量， $1 \leq S_i \leq N-K$ ，然后跟 S_i 个整数，每个整数之间用空格隔开，表示该文件按自然顺序在磁盘上占用的存储块的标识。所有这些数都介于 1 和 N 之间，包括 1 和 N 。一个磁盘说明中所有存储块的标识都是不同的，并且该盘至少有一个空的存储块。

【输出】

对于每一个磁盘说明，只需输出一行句子 “We need M move operations”， M 表示将文件按最佳方式存储到磁盘上所需进行的最少存储块移动操作次数。如果文件已按最佳方式存储，仅需输出 “No optimization needed.”。

【样例】

defrag.in	defrag.out
20 3	We need 9move operations
4 2 3 11 12	
1 7	
3 18 5 10	

【问题分析】

本题可以看作是一个置换的调整问题。按文件块的目标位置对文件块编号后，调整文件块位置的问题就转化为调整置换的问题了。不过，本题还有一点特殊之处，就是存在一些数，它们不要求归位，这些数可作为交换空间。

找出所有的循环节，分情况讨论。如果一个循环节上所有的数都是要求归位的，那么必须先利用交换空间将一个数先交换出去，然后其他数都归位，最后再把交换出去的数换回来。

如果这个循环节上有不需要归位的数，则可直接贪心，将可以归位的尽量归位即可。

9.6 欧几里德的游戏

源程序名	game.??? (pas, c, cpp)
可执行文件名	game.exe
输入文件名	game.in
输出文件名	game.out

【问题描述】

欧几里德的两个后代 Stan 和 Ollie 正在玩一种数字游戏，这个游戏是他们的祖先欧几里德发明的。给定两个正整数 M 和 N ，从 Stan 开始，从其中较大的一个数，减去较小的数的正整数倍，当然，得到的数不能小于 0。然后是 Ollie，对刚才得到的数，和 M ， N 中较小的那个数，再进行同样的操作……直到一个人得到了 0，他就取得了胜利。下面是他们用 (25, 7) 两个数游戏的过程：

Start: 25 7

Stan: 11 7

Ollie: 4 7

Stan: 4 3

Ollie: 1 3

Stan: 1 0

Stan 赢得了游戏的胜利。

现在，假设他们完美地操作，谁会取得胜利呢？

【输入】

第一行为测试数据的组数 C 。下面有 C 行，每行为一组数据，包含两个正整数 M , N 。
(M , N 不超过长整型。)

【输出】

对每组输入数据输出一行，如果 Stan 胜利，则输出 “Stan wins”；否则输出 “Ollie wins”

【样例】

game.in	game.out
2	Stan wins
25 7	Ollie wins
24 15	

【问题分析】

这是一道博弈题。解题的关键在于把握胜负状态的关系。

任意的状态只有两种可能：一种可能是胜状态——即有必胜策略，另一种可能是负状态——即没有必胜策略。对于任意的胜状态，无论如何走，都不可能走到另一个胜状态；而任意的负状态，至少存在一种走法可以走到胜状态。(0, 0) 是初始的胜状态。

考察任意的状态 (a, b) ，不妨假设 $a \geq b$ 。如果 $\frac{a}{b} < 2$ ，则只有一种走法，即将 (a, b) 变为 $(a-b, b)$ 。那么， (a, b) 是何种状态就取决于 $(a-b, b)$ 是何种状态：根据前面胜负状态的定义可知， $(a-b, b)$ 为胜状态时， (a, b) 为负状态； $(a-b, b)$ 为负状态时， (a, b) 为胜状态。

如果 $\frac{a}{b} \geq 2$ ，则至少存在两种走法：将 (a, b) 变为 (c, b) 或 $(c+b, b)$ ，这里 $c = a \bmod b$ 。如果这两个状态中至少有一个是负状态，则根据定义 (a, b) 是胜状态。如果两个状态都是胜状

态，由于 $(c+b, b)$ 可以变为 (c, b) ，这就与“胜状态只能走到负状态”产生了矛盾，所以两个状态都是胜状态的情况是不存在的。因此， $\frac{a}{b} \geq 2$ 时， (a, b) 必为胜状态。

总结一下前面的结论，设 $f(a, b)$ 为状态 (a, b) 的胜负情况 (1 表示胜状态, 0 表示负状态)， $a \geq b$ ，则：

$$1、\frac{a}{b} < 2: f(a, b) = 1 - f(b, a - b) \qquad 2、\frac{a}{b} \geq 2: f(a, b) = 1$$

9.7 百事世界杯之旅

源程序名	pepsl.??? (pas, c, cpp)
可执行文件名	pepsl.exe
输入文件名	pepsl.in
输出文件名	pepsl.out

【问题描述】

“……在 2002 年 6 月之前购买的百事任何饮料的瓶盖上都会有一个百事球星的名字。只要凑齐所有百事球星的名字，就可参加百事世界杯之旅的抽奖活动，获得球星背包，随声听，更克赴日韩观看世界杯。还不赶快行动！”

你关上电视，心想：假设有 n 个不同的球星名字，每个名字出现的概率相同，平均需要买几瓶饮料才能凑齐所有的名字呢？

【输入】

整数 n ($2 \leq n \leq 33$)，表示不同球星名字的个数。

【输出】

输出凑齐所有的名字平均需要买的饮料瓶数。如果是一个整数，则直接输出，否则应该直接按照分数格式输出，例如五又二十分之三应该输出为：

3
5—
20

第一行是分数部分的分子，第二行首先是整数部分，然后是由减号组成的分数线，第三行是分母。减号的个数应等于分母的为数。分子和分母的首位都与第一个减号对齐。

分数必须是不可约的。

【样例】

pepsi.in	pepsi.out
2	3

【提示】

“平均”的定义：如果在任意多次随机实验中，需要购买 $k_1, k_2, k_3, \dots$ 瓶饮料才能凑齐，而 $k_1, k_2, k_3, \dots$ 出现的频率分别是 $p_1, p_2, p_3, \dots$ ，那么，平均需要购买的饮料瓶数应为： $k_1 * p_1 + k_2 * p_2 + k_3 * p_3 + \dots$

【算法分析】

这道题目主要考察的是数学的推导能力。假设 $f[n, k]$ 为一共有 n 个球星，现在还剩 k 个未收集到，还需购买饮料的平均次数。则有：

$$f(n, k) = \frac{(n-k)f(n, k)}{n} + \frac{kf(n, k-1)}{n} + 1$$

经移项整理，可得：

$$f(n, k) = f(n, k-1) + \frac{n}{k}$$

我们所要求的实际上应该是 $f(n, n)$ ，根据 $f(n, k)$ 的递推式，可得

$$f(n, n) = n \sum_{k=1}^n \frac{1}{k} = nH(n)$$

其中， $H(n)$ 表示 Harmonic Number。

9.8 倒酒

源程序名	pour.??? (pas, c, cpp)
可执行文件名	pour.exe
输入文件名	pour.in
输出文件名	pour.out

【问题描述】

Winy 是一家酒吧的老板，他的酒吧提供两种体积的啤酒， a ml 和 b ml，分别使用容积为 a ml 和 b ml 的酒杯来装载。

酒吧的生意并不好。Winy 发现酒鬼们都非常穷。有时，他们会因为负担不起 a ml 或者 b ml 啤酒的消费，而不得不离去。因此，Winy 决定出售第三种体积的啤酒（较小体积的啤酒）。

Winy 只有两种杯子，容积分别为 a ml 和 b ml，而且啤酒杯是没有刻度的。他只能通过两种杯子和酒桶间的互相倾倒来得到新的体积的酒。

为了简化倒酒的步骤，Winy 规定：

- (1) $a \geq b$;
- (2) 酒桶容积无限大，酒桶中酒的体积也是无限大（但远小于桶的容积）；
- (3) 只包含三种可能的倒酒操作：
 - ①将酒桶中的酒倒入容积为 b ml 的酒杯中；
 - ②将容积为 a ml 的酒杯中的酒倒入酒桶；
 - ③将容积为 b ml 的酒杯中的酒倒入容积为 a ml 的酒杯中。
- (4) 每次倒酒必须把杯子倒满或把被倾倒的杯子倒空。

Winy 希望通过若干次倾倒得到容积为 a ml 酒杯中剩下的酒的体积尽可能小，他请求你帮助他设计倾倒的方案

【输入】

两个整数 a 和 b ($0 < b \leq a \leq 10^9$)

【输出】

第一行一个整数 c ，表示可以得到的酒的最小体积。

第二行两个整数 P_a 和 P_b （中间用一个空格分隔），分别表示从体积为 a ml 的酒杯中倒出酒的次数和将酒倒入体积为 b ml 的酒杯中的次数。

若有多种可能的 P_a 、 P_b 满足要求，那么请输出 P_a 最小的一个。若在 P_a 最小的情况下，有多个 P_b 满足要求，请输出 P_b 最小的一个。

【样例】

pour.in	pour.out
5 3	1

倾倒的方案为：

- 1、桶→B 杯； 2、B 杯→A 杯；
- 3、桶→B 杯； 4、B 杯→A 杯；
- 5、A 杯→桶； 6、B 杯→A 杯；

【问题分析】

解模方程。

首先， $c = \gcd(a, b)$ 。写成模方程形式即为 $bx \equiv c \pmod{a}$ 。

设 P_a, P_b 分别为从体积为 a mL 的酒杯中倒出酒的次数和将酒倒入体积为 b mL 的酒杯中的次数，则有 $c = bP_b - aP_a$ 。

用 Euclid 辗转相除法求出 P_a, P_b 即可。时间复杂度为 $O(\log_2 n)$ 。

9.9 班级聚会

源程序名	reunion.??? (pas, c, cpp)
可执行文件名	reunion.exe
输入文件名	reunion.in
输出文件名	reunion.out

【问题描述】

毕业 25 年以后，我们的主人公开始准备同学聚会。打了无数电话后他终于搞到了所有同学的地址。他们有些人仍在本城市，但大多数人分散在其他的城市。不过，他发现一个巧合，所有地址都恰好分散在一条铁路线上。他准备出发邀请但无法决定应该在哪个地方举行宴会。最后他决定选择一个地点，使大家旅行的花费和最小。

不幸的是，我们的主人公既不擅长数学，也不擅长计算机。他请你帮忙写一个程序，根据他同学的地址，选择聚会的最佳地点。

【输入】

输入文件的每一行描述了一个城市的信息。

首先是城市里同学的个数，紧接着是这个城市到 Moscow（起点站）的距离（km），最后是城市的名称。最后一行描述的总是 Moscow，它在铁路线的一端，距离为 0。

【输出】

聚会地点城市名称和旅行费用（单程），两者之间用一空格隔开。每 km 花费一个卢布。

【样例】

reunion.in	reunion.out
7 9289 Vladivostok	Yalutorovsk 112125
5 8523 Chabarovsk	
3 5184 Irkutsk	
8 2213 Yalutorovsk	
10 0 Moscow	

【问题分析】

中位数问题。找这样一个点（城市）——它左边的人数与右边的人数差的绝对值最小。时间复杂度为 $O(n)$ 。

本问题的参考程序作者特意不加注释，以考察读者能否根据以上问题分析，将程序看懂并理解。